Self Healing Data Pipeline (Work Report)

I have been working on building a Self-Healing Data Pipeline system that focuses on automatically detecting and repairing failures caused by schema drift. In modern data systems, schema drift is one of the most common issues and contributes to nearly 40% of pipeline failures. My system is designed to address this problem by combining metadata-driven design, log-based detection, and LLM-powered repair mechanis**ms**.

Metadata Representation

The core of the system is a **metadata graph representation** of the data sources.

- Each database is represented as a node
- It has edges like **"HAS_TABLE"**, connecting it to tables
- Each table node is further connected to its columns

This creates a hierarchical graph:

Database → Tables → Columns

This structure allows the system to:

- Track schema changes efficiently
- Maintain relationships between entities
- Apply updates in a structured and scalable way

Schema Drift Detection

Schema drift occurs when:

- Column names change
- Tables are altered
- Data types are modified

To detect these changes:

- The system parses logs generated from heterogeneous data sources
- It filters specifically for **DDL operations** (like ALTER, DROP, RENAME)
- Since not all databases support DDL triggers (e.g., some support like Postgres, others don't), the system aggregates logs from multiple sources

Incremental Metadata Updation

Instead of reprocessing everything, the system follows incremental update approach:

- Metadata contains a last_updated timestamp
- Only logs generated after this timestamp are fetched
- This reduces computation and ensures efficiency

Steps:

1. Fetch logs after last_updated
2. Extract relevant DDL operations
3. Process schema drift changes
4. Update metadata graph

LLM-Based Repair Using DAG

One of the key innovations in the system is using an LLM to **generate a DAG (Directed Acyclic Graph)** representing schema changes.

Instead of manually defining transformation rules, the **parsed logs are directly provided to the LLM**, which interprets schema evolution and constructs the DAG automatically.
This approach is more scalable and adaptable, as it can generalize across diverse schema changes without requiring hardcoded logic for every possible transformation.

Example:

If a column evolves like this:

name → full_name → his_name → employee_name

The system constructs a DAG:

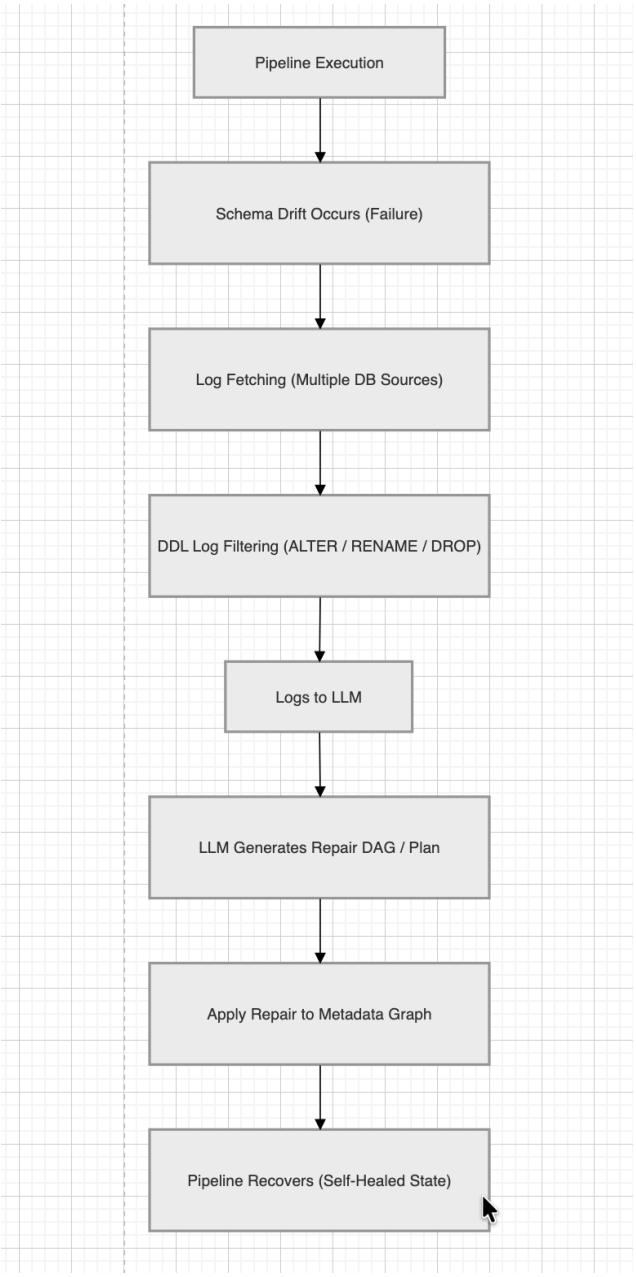
name → full_name → his_name → employee_name

Instead of applying multiple step-by-step updates, the LLM intelligently:

- Understands the full transformation chain
- Directly maps:

name → employee_name

Current System Architecture



Components:

Data Sources

- Multiple heterogeneous databases (Postgres, MySQL, etc.)

Log Parser

- Extracts DDL operations
- Filters relevant schema changes

Metadata Graph

- Stores schema structure
- Maintains relationships

LLM Engine

- Takes parsed logs as input
- Generates DAG of schema transformations
- Optimizes update paths

Schema Repair Engine

- Applies DAG updates
- Updates metadata incrementally

Extension: HoloClean (VLDB) Integration

To further enhance the system, I explored the HoloClean (VLDB) paper:

"HoloClean: Holistic Data Repairs with Probabilistic Inference"

Key Idea:

HoloClean focuses on **data-level error correction**, not just schema-level fixes.

It uses:

- Probabilistic inference
- Statistical models
- Constraint-based validation

What makes it interesting:

- Detects inconsistencies in data
- Repairs incorrect values using learned patterns
- Combines machine learning + rules + constraints

Planned Integration with Current System

Currently, my system handles:

- Schema drift detection
- Metadata repair

With HoloClean integration, it will extend to:

Data-Level Self-Healing

- Detect incorrect or inconsistent data
- Apply probabilistic corrections
- Improve data quality before pipeline execution

End Goal

A fully autonomous pipeline that can:

- Detect schema issues
- Repair metadata
- Clean and correct data
- Maintain consistency across sources

Conclusion

So far, I have successfully implemented:

- Metadata graph representation
- Log-based schema drift detection
- Incremental update mechanism
- LLM-based DAG generation for schema repair

Currently, I am working on:

- Integrating HoloClean-based probabilistic data repair
- Extending the system from schema-level healing → full data-level healing